

ESP-IDF

Table of Contents

1. [What is ESP-IDF](#)
2. [Project structure](#)
3. [Component system](#)
4. [Build system — CMake and idf.py](#)
5. [menuconfig — configuring the project](#)
6. [app_main and the entry point](#)
7. [Logging](#)
8. [Non-volatile storage \(NVS\)](#)
9. [GPIO](#)
10. [UART](#)
11. [Event loop](#)
12. [Error handling](#)
13. [Flashing and monitoring](#)
14. [Partitions](#)
15. [SmartDB component dependency map](#)
16. [Common mistakes](#)
17. [Glossary](#)

What is ESP-IDF

ESP-IDF (Espressif IoT Development Framework) is the official development framework for ESP32 microcontrollers. It provides:

- A **build system** (CMake + Ninja) that compiles your C/C++ code for ESP32
- A **hardware abstraction layer (HAL)** — APIs for GPIO, UART, I2C, SPI, WiFi, Bluetooth, and every other ESP32 peripheral
- **FreeRTOS** — already integrated, every ESP-IDF project runs FreeRTOS
- **MBEDTLS** — already integrated, used for DTLS in SmartDB
- **Component system** — modular packaging of reusable code
- **menuconfig** — Kconfig-based configuration system

ESP-IDF is written in C. You write your firmware in C (or C++ with some limitations). Python is used only for build tooling — not in the firmware itself.

Version used in SmartDB

Check the current version:

```
idf.py --version
```

SmartDB targets **ESP-IDF v5.x** — verify your installed version matches. API changes between major versions (v4 → v5) can break existing code.

Project structure

A minimal ESP-IDF project:

```
my_project/
├── CMakeLists.txt           ← top-level build file
├── main/
│   ├── CMakeLists.txt      ← main component build file
│   └── main.c              ← entry point (app_main lives here)
└── components/             ← optional: your custom components
    └── my_component/
        ├── CMakeLists.txt
        ├── include/
        │   └── my_component.h
        └── my_component.c
```

Top-level CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(smartdb_firmware)
```

This is almost always identical across projects — just change the project name.

main/CMakeLists.txt

```
idf_component_register(
    SRCS "main.c"
    INCLUDE_DIRS "."
)
```

Component system

The component system is ESP-IDF's way of organizing modular, reusable code. Each component is a directory with:

- A **CMakeLists.txt** declaring its source files and dependencies
- An **include/** directory with its public header files
- One or more **.c** implementation files

Why components matter for SmartDB

SmartDB's modular firmware structure (sensors, safety_layer, protocol, dtls_transport, node_fsm, etc.) maps directly to ESP-IDF components. Each subsystem is its own component — independently buildable, testable, and replaceable.

Component CMakeLists.txt

```
idf_component_register(  
    SRCS  
        "sensor_sampling.c"  
        "ads1115.c"  
        "mlx90640.c"  
    INCLUDE_DIRS  
        "include"  
    REQUIRES  
        driver          # ESP-IDF driver component (I2C, GPIO, UART)  
        freertos        # FreeRTOS  
        esp_log         # Logging  
        i2c_protocol    # Another SmartDB component (custom)  
)
```

SRCS — list all .c files in this component **INCLUDE_DIRS** — directories containing public headers **REQUIRES** — components this component depends on

ESP-IDF resolves the dependency graph at build time — if **safety_layer** requires **sensors**, and **sensors** requires **driver**, ESP-IDF ensures **driver** is compiled before both. You don't manage compile order manually.

ESP-IDF built-in components

Commonly used built-in components:

Component	What it provides
driver	I2C, SPI, UART, GPIO, ADC drivers
freertos	FreeRTOS tasks, queues, semaphores
esp_log	Logging macros (ESP_LOGI, ESP_LOGE, etc.)
esp_wifi	WiFi stack
esp_netif	Network interface abstraction
nvs_flash	Non-volatile storage
mbedtls	TLS/DTLS, AES, HMAC
esp_timer	High-resolution timer
esp_system	System info, restart, heap monitoring

Build system

idf.py commands

```
# Configure the project (opens menuconfig UI)
idf.py menuconfig

# Build the firmware
idf.py build

# Flash to connected ESP32 (replace /dev/ttyUSB0 with your port)
idf.py -p /dev/ttyUSB0 flash

# Open serial monitor
idf.py -p /dev/ttyUSB0 monitor

# Build, flash, and monitor in one command (most common during development)
idf.py -p /dev/ttyUSB0 flash monitor

# Clean build directory
idf.py fullclean
```

Finding your ESP32's serial port

On Ubuntu:

```
ls /dev/ttyUSB* # most common for CH340-based ESP32 boards
ls /dev/ttyACM* # alternative
dmesg | tail -20 # shows recent USB events when you plug in ESP32
```

Build output

Successful build produces:

```
build/
├─ smartdb_firmware.bin    ← main firmware binary
├─ bootloader/
│   └─ bootloader.bin      ← ESP32 bootloader
└─ partition_table/
    └─ partition-table.bin ← flash partition layout
```

All three are flashed to the ESP32 during `idf.py flash`.

menuconfig

menuconfig is ESP-IDF's configuration system — a text-based UI where you enable/disable features, set parameters, and configure hardware.

```
idf.py menuconfig
```

Navigate with arrow keys, Enter to enter a submenu, Space to toggle, Q to quit and save.

Key settings for SmartDB

WiFi:

```
Component config → ESP32-specific → WiFi
```

Default settings are fine — SmartDB uses standard station mode.

MBEDTLS hardware AES acceleration:

```
Component config → mbedTLS → Hardware acceleration → Enable hardware AES
```

Must be enabled for efficient encryption on ESP32.

FreeRTOS stack overflow detection:

```
Component config → FreeRTOS → Check for stack overflow → Method 2
```

Enable during development — disable in production if stack sizes are confirmed.

Serial port (for logging and flashing):

```
Component config → ESP System Settings → UART for console output
```

Usually UART0 by default — matches the USB-serial on the ESP32-DEVKIT-V1.

Partition table:

```
Partition Table → Partition Table
```

Set to "Custom partition table CSV" if using a custom partition layout

Saving configuration

menuconfig saves to `sdkconfig` in the project root. This file is generated — commit it to git so the team uses the same configuration. Never edit `sdkconfig` manually — always use menuconfig.

app_main

`app_main()` is the entry point for your application. It runs as a FreeRTOS task (the "main task") on startup. Unlike `main()` in desktop C, it must **not return** — if it returns, the main task is deleted.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "nvs_flash.h"

// Include SmartDB component headers
#include "node_fsm.h"
#include "safety_layer.h"
#include "sensor_sampling.h"
#include "dtls_transport.h"
#include "status_led.h"
#include "uart_jetson.h"

static const char *TAG = "app_main";

void app_main(void) {
    // 1. Initialize NVS (required before WiFi and most other components)
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
        ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        nvs_flash_erase();
        nvs_flash_init();
    }

    // 2. Initialize hardware
    i2c_init();
    relay_control_init();
    status_led_init();
    uart_jetson_init();

    // 3. Create inter-task communication objects
    // (queues, semaphores, mutexes – created before tasks that use them)
    sensor_queue = xQueueCreate(5, sizeof(sensor_data_t));
    override_queue = xQueueCreate(1, sizeof(override_cmd_t));
    incoming_queue = xQueueCreate(10, sizeof(protocol_msg_t));
    i2c_mutex = xSemaphoreCreateMutex();
    breach_semaphore = xSemaphoreCreateBinary();

    // 4. Start tasks (see FreeRTOS knowledge base for priorities and
    cores)
    safety_layer_start_task();
    sensor_sampling_start_task();
    dtls_rx_start_task();
    telemetry_start_task();
    node_fsm_start_task();
    status_led_start_task();
    uart_jetson_start_task();
}
```

```

    ESP_LOGI(TAG, "SmartDB firmware started – all tasks running");

    // 5. app_main can now delete itself or loop forever doing nothing
    // Deleting itself is cleaner – frees the main task stack
    vTaskDelete(NULL);
}

```

Why app_main deletes itself

The main task has a default stack size (configurable in menuconfig). Once all FreeRTOS tasks are created, app_main's work is done. Deleting it frees that stack memory back to the heap — important on a memory-constrained device like ESP32.

Logging

ESP-IDF provides a logging system with five levels:

```

#include "esp_log.h"

static const char *TAG = "my_component"; // tag appears in log output

ESP_LOGE(TAG, "Error: %d", error_code); // Error – always shown
ESP_LOGW(TAG, "Warning: value=%f", val); // Warning
ESP_LOGI(TAG, "Info: started"); // Info
ESP_LOGD(TAG, "Debug: raw=%d", raw); // Debug – verbose
ESP_LOGV(TAG, "Verbose: byte=%02X", b); // Verbose – very verbose

```

Output format:

```

I (1234) sensor_sampling: Irms=5.23A, THD=3.1%
| | |
| | └─ TAG
| └─ timestamp (ms since boot)
└─ level (E/W/I/D/V)
    └─ message

```

Setting log level

In menuconfig:

Component config → Log output → Default **log** verbosity

Or at runtime (useful for debugging specific components without rebuilding):

```
esp_log_level_set("sensor_sampling", ESP_LOG_DEBUG); // verbose for this
tag
esp_log_level_set("dtls_transport", ESP_LOG_WARN); // quiet for this tag
esp_log_level_set("*", ESP_LOG_INFO); // default for all
```

SmartDB logging convention

Use consistent TAGs matching component names:

```
// In each component's .c file:
static const char *TAG = "safety_layer"; // matches component directory
name
static const char *TAG = "sensor_sampling";
static const char *TAG = "dtls_transport";
static const char *TAG = "node_fsm";
```

NVS (Non-Volatile Storage)

NVS is ESP32's key-value store in flash — persists across reboots. SmartDB uses it to store:

- PSK (pre-shared key) for DTLS — must survive reboots
- Last known configuration (thresholds, sampling rate) — survive reboots
- Device ID — fixed at manufacture time

Basic NVS usage

```
#include "nvs_flash.h"
#include "nvs.h"

// Writing
nvs_handle_t handle;
nvs_open("smartdb", NVS_READWRITE, &handle);
nvs_set_blob(handle, "psk", psk_data, psk_len);
nvs_set_u32(handle, "device_id", 0x00000001);
nvs_commit(handle);
nvs_close(handle);

// Reading
nvs_open("smartdb", NVS_READONLY, &handle);
size_t psk_len = sizeof(psk_buffer);
nvs_get_blob(handle, "psk", psk_buffer, &psk_len);
nvs_close(handle);
```

NVS namespaces

NVS is organized into namespaces — like folders. SmartDB uses one namespace: `"smartdb"`. All keys are stored within this namespace.

NVS encryption

Enable NVS encryption in menuconfig to protect the PSK at rest:

Security features → Enable NVS Encryption

This encrypts the entire NVS partition using a key stored in eFuse — the PSK cannot be read by physically dumping the flash chip.

GPIO

GPIO (General Purpose Input/Output) controls digital pins on ESP32.

Basic GPIO usage

```
#include "driver/gpio.h"

#define RELAY_PIN    GPIO_NUM_18
#define LED_R_PIN    GPIO_NUM_25
#define LED_G_PIN    GPIO_NUM_26
#define LED_B_PIN    GPIO_NUM_27

// Configure as output
gpio_config_t io_conf = {
    .pin_bit_mask = (1ULL << RELAY_PIN) |
                    (1ULL << LED_R_PIN) |
                    (1ULL << LED_G_PIN) |
                    (1ULL << LED_B_PIN),
    .mode          = GPIO_MODE_OUTPUT,
    .pull_up_en    = GPIO_PULLUP_DISABLE,
    .pull_down_en  = GPIO_PULLEDOWN_DISABLE,
    .intr_type     = GPIO_INTR_DISABLE,
};
gpio_config(&io_conf);

// Set output level
gpio_set_level(RELAY_PIN, 0); // LOW = relay open (safe state)
gpio_set_level(RELAY_PIN, 1); // HIGH = relay closed (power on)
gpio_set_level(LED_R_PIN, 1); // LED on
```

GPIO interrupt (for ADS1115 ALERT/RDY pin)

```
#define ALERT_PIN    GPIO_NUM_34 // input only pin on ESP32
```

```
// ISR handler
static void IRAM_ATTR alert_isr_handler(void *arg) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xSemaphoreGiveFromISR(alert_semaphore, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

// Setup
gpio_config_t alert_conf = {
    .pin_bit_mask = (1ULL << ALERT_PIN),
    .mode          = GPIO_MODE_INPUT,
    .pull_up_en    = GPIO_PULLUP_ENABLE,
    .intr_type     = GPIO_INTR_NEGEDGE,    // trigger on falling edge
};
gpio_config(&alert_conf);
gpio_install_isr_service(0);
gpio_isr_handler_add(ALERT_PIN, alert_isr_handler, NULL);
```

UART

SmartDB uses UART2 to communicate with the Jetson Nano over the TXS0104E level shifter (J8 connector).

```
#include "driver/uart.h"

#define UART_PORT      UART_NUM_2
#define UART_TX_PIN    GPIO_NUM_17
#define UART_RX_PIN    GPIO_NUM_16
#define UART_BAUD_RATE 921600          // must match Jetson side
#define UART_BUF_SIZE  2048

esp_err_t uart_jetson_init(void) {
    uart_config_t uart_config = {
        .baud_rate    = UART_BAUD_RATE,
        .data_bits    = UART_DATA_8_BITS,
        .parity        = UART_PARITY_DISABLE,
        .stop_bits    = UART_STOP_BITS_1,
        .flow_ctrl     = UART_HW_FLOWCTRL_DISABLE,
    };
    uart_param_config(UART_PORT, &uart_config);
    uart_set_pin(UART_PORT, UART_TX_PIN, UART_RX_PIN,
        UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);
    return uart_driver_install(UART_PORT, UART_BUF_SIZE, UART_BUF_SIZE,
        0, NULL, 0);
}

// Send a thermal frame to Jetson
esp_err_t uart_send_frame(uint8_t *frame_data, size_t len) {
    int written = uart_write_bytes(UART_PORT, frame_data, len);
    return (written == len) ? ESP_OK : ESP_FAIL;
}
```

```
// Receive anomaly score from Jetson
esp_err_t uart_receive_score(float *score) {
    uint8_t buf[4];
    int len = uart_read_bytes(UART_PORT, buf, 4, pdMS_TO_TICKS(50));
    if (len != 4) return ESP_ERR_TIMEOUT;
    memcpy(score, buf, 4); // float is 4 bytes
    return ESP_OK;
}
```

Event loop

ESP-IDF has a default event loop for system events (WiFi connected, disconnected, IP obtained, etc.). SmartDB uses it to handle WiFi events during the DISCOVERY state.

```
#include "esp_event.h"
#include "esp_wifi.h"

static void wifi_event_handler(void *arg, esp_event_base_t event_base,
                               int32_t event_id, void *event_data) {
    if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_START) {
        esp_wifi_connect();
    } else if (event_base == WIFI_EVENT &&
               event_id == WIFI_EVENT_STA_DISCONNECTED) {
        ESP_LOGW(TAG, "WiFi disconnected – retrying");
        esp_wifi_connect();
        // Notify FSM task to transition back to DISCOVERY state
        xTaskNotify(fsm_task_handle, FSM_EVENT_WIFI_LOST, eSetBits);
    } else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t *event = (ip_event_got_ip_t *)event_data;
        ESP_LOGI(TAG, "Got IP: " IPSTR, IP2STR(&event->ip_info.ip));
        // Notify FSM task that WiFi is ready – can start DTLS handshake
        xTaskNotify(fsm_task_handle, FSM_EVENT_WIFI_READY, eSetBits);
    }
}

// Register handlers
esp_event_loop_create_default();
esp_event_handler_register(WIFI_EVENT, ESP_EVENT_ANY_ID,
                           wifi_event_handler, NULL);
esp_event_handler_register(IP_EVENT, IP_EVENT_STA_GOT_IP,
                           wifi_event_handler, NULL);
```

Error handling

ESP-IDF functions return `esp_err_t` — an integer error code. Always check return values.

Error codes

Code	Value	Meaning
ESP_OK	0	Success
ESP_FAIL	-1	Generic failure
ESP_ERR_NO_MEM	0x101	Out of memory
ESP_ERR_INVALID_ARG	0x102	Invalid argument
ESP_ERR_TIMEOUT	0x107	Operation timed out
ESP_ERR_NOT_FOUND	0x105	Resource not found

Error checking macros

ESP-IDF provides convenience macros:

```
// ESP_ERROR_CHECK: if err != ESP_OK, logs error and aborts (resets ESP32)
// Use only for truly unrecoverable errors (hardware init failures)
ESP_ERROR_CHECK(i2c_driver_install(I2C_PORT, I2C_MODE_MASTER, 0, 0, 0));

// For recoverable errors, check manually:
esp_err_t ret = ads1115_read(&value);
if (ret != ESP_OK) {
    ESP_LOGW(TAG, "ADS1115 read failed: %s – skipping cycle",
              esp_err_to_name(ret));
    return; // skip this sample, try again next cycle
}
```

esp_err_to_name

Converts an error code to a human-readable string — use in log messages:

```
ESP_LOGE(TAG, "I2C failed: %s", esp_err_to_name(ret));
// Output: I2C failed: ESP_ERR_TIMEOUT
```

Flashing and monitoring

Wiring for flashing

ESP32-DEVKIT-V1 connects via USB — the onboard CH340 USB-serial chip handles the connection. Just plug in USB.

Putting ESP32 in flash mode

ESP32-DEVKIT-V1 enters flash mode automatically when `idf.py flash` is run (using DTR/RTS control signals from the USB-serial chip). If automatic mode fails, hold the BOOT button while pressing EN to manually enter flash mode.

Flash command

```
idf.py -p /dev/ttyUSB0 flash monitor
```

Serial monitor

`idf.py monitor` opens the serial monitor at 115200 baud (default ESP-IDF console baud rate). Press Ctrl+] to exit.

The monitor also decodes panic backtraces — if ESP32 crashes, it prints a backtrace that monitor decodes into function names and line numbers:

```
Guru Meditation Error: Core 0 panic'ed (LoadProhibited)
Backtrace: 0x40082abc:0x3fffb1234 0x40081234:0x3fffb1240
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
           monitor decodes this → safety_layer.c:87 in safety_check()
```

Partitions

ESP32 flash is divided into partitions — named regions with specific purposes. The default partition table for SmartDB:

#	Name	Type	SubType	Offset	Size	
nvs		data	nvs	0x9000	0x6000	# NVS storage (PSK, config)
phy_init		data	phy	0xf000	0x1000	# WiFi calibration data
factory		app	factory	0x10000	0x300000	# Main firmware

Custom partition table

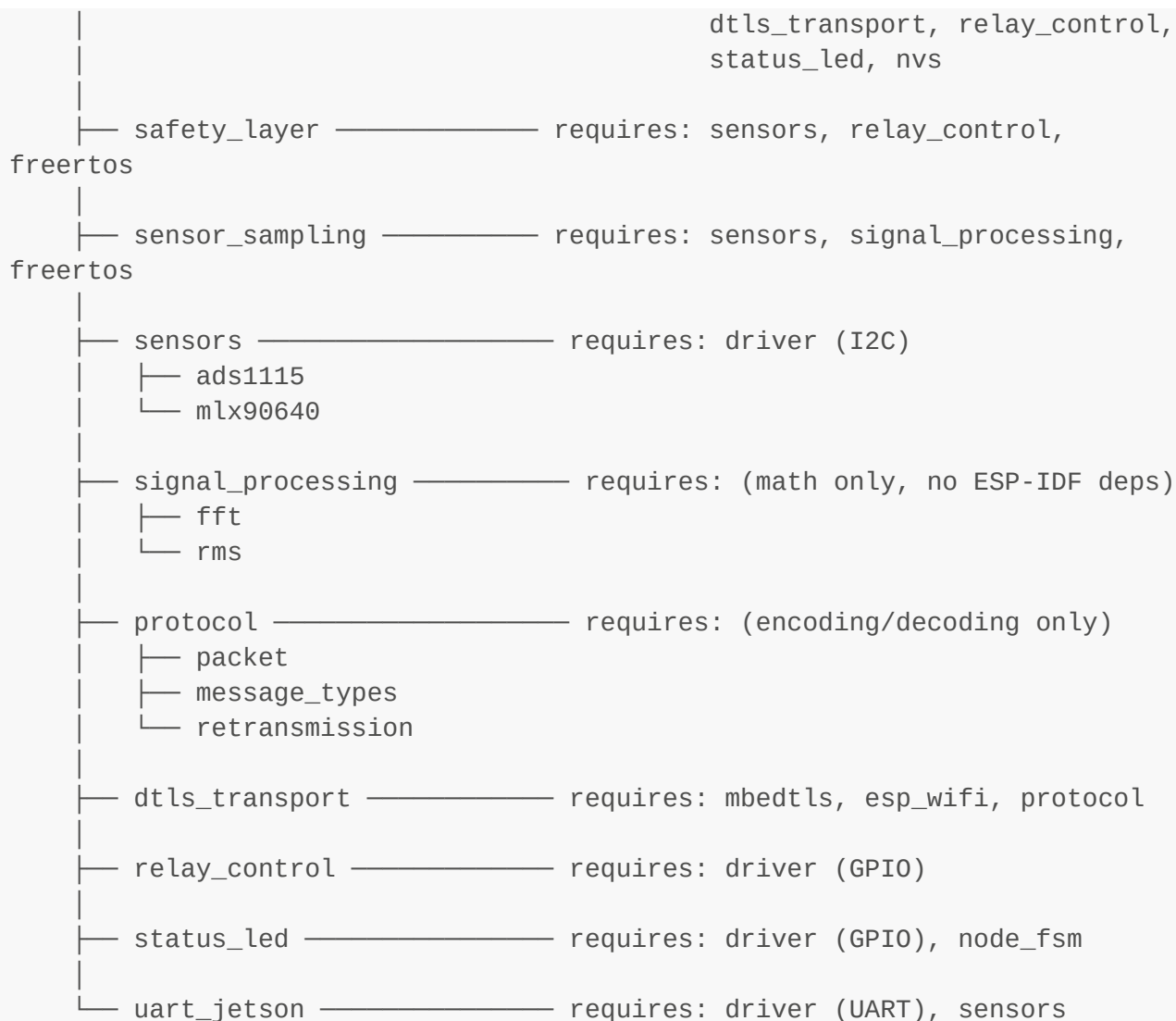
Create `partitions.csv` in the project root, then set in menuconfig:

Partition Table → Custom partition table CSV

If you need OTA (over-the-air firmware updates) later, you'll need two app partitions (ota_0, ota_1) — a significant structural change worth planning for now even if not implementing immediately.

SmartDB component dependency map

```
app_main
|
├── node_fsm ─────────── requires: safety_layer, protocol,
```



Build order is resolved automatically by ESP-IDF from this dependency graph. You never specify build order manually.

Common mistakes

1. Calling `idf.py` before running `export.sh`

```

# WRONG – idf.py not found
idf.py build

# CORRECT – run this first in every new terminal
. ~/esp/esp-idf/export.sh
# or use the alias: get_idf

```

2. Forgetting `nvs_flash_init()` before WiFi WiFi requires NVS. Not initializing NVS first causes a panic on boot. Always call `nvs_flash_init()` at the top of `app_main`.

3. Using `ESP_ERROR_CHECK` on recoverable errors `ESP_ERROR_CHECK` calls `abort()` on failure — resetting the ESP32. Use it only for hardware init failures where the system cannot function. For runtime errors (I2C read failure, DTLS packet loss), check manually and handle gracefully.

- 4. Not calling `idf_component_register` correctly** Missing a `REQUIRES` entry means your component can compile but fails to link, with cryptic "undefined reference" errors. Check `REQUIRES` when you see linker errors.
- 5. Editing `sdkconfig` directly** `sdkconfig` is generated by `menuconfig`. Manual edits get overwritten the next time `menuconfig` runs. Always use `idf.py menuconfig` to change configuration.
- 6. Stack size too small for mbedTLS tasks** mbedTLS uses significant stack during DTLS handshake — up to 8KB. Any task that calls mbedTLS functions needs at least 8192 bytes stack. See FreeRTOS knowledge base for stack sizing guidelines.
- 7. Creating tasks before queues they use** If a task starts and immediately tries to use a queue that hasn't been created yet, it will crash. Always create all FreeRTOS objects (queues, semaphores, mutexes) before calling `xTaskCreate`.
- 8. Not enabling hardware AES in `menuconfig`** Software AES on ESP32 is 3x slower than hardware AES. If you forget to enable hardware acceleration in `menuconfig`, encryption will noticeably impact your 100ms sampling duty cycle.

Glossary

Term	Meaning
<code>app_main</code>	Entry point function for ESP-IDF applications
Boot button	Physical button on ESP32-DEVKIT-V1 for manual flash mode entry
CH340	USB-serial chip on ESP32-DEVKIT-V1 for PC communication
CMake	Build system generator used by ESP-IDF
Component	Modular unit of ESP-IDF code with its own <code>CMakeLists.txt</code>
eFuse	One-time-programmable memory on ESP32 for encryption keys
EN button	Reset button on ESP32-DEVKIT-V1
<code>esp_err_t</code>	Integer type for ESP-IDF error codes
<code>ESP_ERROR_CHECK</code>	Macro that aborts on non- <code>ESP_OK</code> return value
ESP-IDF	Espressif IoT Development Framework
Event loop	System for handling WiFi, IP, and other system events
Flash	ESP32's non-volatile storage (4MB on DEVKIT-V1)
HAL	Hardware Abstraction Layer — hardware-independent API
<code>IDF_PATH</code>	Environment variable pointing to ESP-IDF installation
<code>idf.py</code>	Command-line tool for building, flashing, monitoring
<code>IRAM_ATTR</code>	Attribute placing a function in internal RAM (required for ISRs)
ISR	Interrupt Service Routine

Term	Meaning
Kconfig	Configuration system used by menuconfig
menuconfig	Text UI for configuring ESP-IDF project settings
Ninja	Fast build tool used by ESP-IDF
NVS	Non-Volatile Storage — key-value store in ESP32 flash
Partition	Named region of ESP32 flash with specific purpose
sdkconfig	Generated configuration file from menuconfig
TAG	String identifying the source of a log message
UART0	Default console UART (USB-serial on DEVKIT-V1)
UART2	UART used for Jetson Nano communication in SmartDB